Accelerated Feature Extraction and Refinement for Improved Aerial Scene Categorization

Xiaohan Tu, Laurence Tianruo Yang, *Fellow, IEEE*, Siping Liu, and Renfa Li, *Senior Member, IEEE*

***Abstract*— Deep learning (DL) has displayed superior performance in aerial scene (AS) categorization. However, existing methods for AS classification tend to lack adaptability and efficiency, particularly in optimizing its performance on different embedded devices. They also often fail to dynamically adjust to varying scales of feature representations, which can limit their effectiveness across different datasets and devices. To solve these issues, we provide two algorithms. The first algorithm explores local key features by mining the interactivity between channels. The range of cross-channel interactions is dynamically determined through an adaptive strategy. This ensures that convolution operations are optimized. The resulting features are improved by the second proposed algorithm. The second algorithm introduces convolutions, attention, and functions to calculate the weight of features. It enhances feature discriminative power by assigning adaptive weights. Then, we introduce an inference acceleration method for AS categorization. We create efficient codes for the proposed algorithms through automated optimization to match different devices. The inference time of the proposed method is reduced on various devices. Experiments on three frequently used datasets show we attain higher accuracy. The accuracy of some categories reaches 100%. In contrast to similar methods, our accelerated algorithm has at least 43.16%, 53.37%, 10.07%, 31.16%, and 8.08% less inference time on RTX 2080Ti, Titan V, Jetson TX2, Jetson NX, and Jetson Nano GPUs, respectively.**

***Index Terms*— Acceleration, aerial scene (AS) classification, deep neural networks (DNNs), feature extraction, feature refinement.**

I. Introduction

AERIAL scenes (ASs) frequently consist of various objects that are jammed together in aerial views. AS classification strives to categorize AS into various semantic classes and allocate corresponding labels to scene images. We introduce the relevant background, motivation, and contribution of AS classification as follows.

A. Background

Recently, the ability to automatically and effectively classify AS has become particularly necessary because of remote sensing (RS) requirements. RS images have performed a vital effect in landslide inventory mapping [1], environmental protection [2], [3], surface cover categorization [4], [5], [6], and monitoring [7], [8], [9], [10]. However, AS classification is a complex task because AS has the following qualities.

1) The AS images are noisy. The AS images are often large-scale. They are captured using aerial devices, such as planes. Thin clouds usually appear in some AS images when the weather is cloudy. Some images may appear shadowed when the altitude angles of the sun are varying.
2) The ground objects in AS exhibit complex and varied configurations. They are increasingly diversified and coexist in different shapes. These ground objects may gather more, and there is no distribution in a certain scene. Therefore, the distribution of objects is irregular.
3) The ground objects in AS are rich, and their backgrounds are cluttered. Thanks to the fast progress of RS technique, the amount of AS images has increased exponentially. Different AS images include background information irrelevant to labels. They also contain a rich variety of ground objects, such as railways, airports, and highways.

Past studies lessened the above three challenges using handcrafted features such as spectral and spatial characteristics. For example, the research [11] exploited textures and spectrum to classify AS. In the handcrafted feature-based approaches, the matrix–vector spaces are constrained. Relative to techniques relying on manually crafted features, deep neural networks (DNNs) extracted more discriminative information. DNNs gain popularity in AS classification because they possess a more robust feature representation.

B. Motivation

Many AS classification techniques rely on DNNs [12], [13], [14], [15], [16], [17], [18], [19]. DNNs hold great promise in enhancing AS features. They explored AS features from original inputs. The features were changed into high-level ones. These models often include nonlinear DNN layers. They

Manuscript received 1 November 2023; revised 4 January 2024, 11 March 2024, and 18 April 2024; accepted 16 May 2024. Date of publication 21 May 2024; date of current version 10 June 2024. This work was supported in part by the Henan Natural Science Foundation under Grant 242300420693, in part by the Key Scientific Research Project Plan of Colleges and Universities in Henan Province under Grant 23A520042, in part by the Fundamental Research Funds for Central Universities of Zhengzhou Police University under Grant 2022TJJBKY002 and Grant 2023TJJBKY012, in part by the Key Research and Development and Promotion Special Project of Henan Province (Scientific and Technological Research) under Grant 232102210022, and in part by the National Natural Science Foundation of China under Grant 61932010. *(Corresponding author: Siping Liu.)*

Xiaohan Tu is with the Department of Image and Network Investigation, Zhengzhou Police University, Zhengzhou 450053, China (e-mail: txhan@hnu.edu.cn).

Laurence Tianruo Yang is with the Department of Computer Science, St. Francis Xavier University, Antigonish, NS B2G 2W5, Canada (e-mail: ltyang@ieee.org).

Siping Liu is with the School of Cyber Science and Engineering, Zhengzhou University, Zhengzhou 450002, China (e-mail: liusiping@hnu.edu.cn).

Renfa Li is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: lirenfa@hnu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2024.3403877

1558-0644 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

need no knowledge of earlier experience and directly capture AS information. They also optimized AS categorization. As an example, Tan and Le [12] exploited structure search and zoom to reduce model volume and training speed. Liu et al. [17] re-explored the design space of convolutional neural networks (CNNs) and important factors that affect performance differences. However, they ignored the reasoning time on different devices. In addition, it is essential to accurately record subtle traits for AS classification. Certain regional characteristics are essential for identifying land cover categories or object types [20], [21], [22]. Currently, there are still common limitations as follows.

1) The existing DNN approaches focus on global information and miss locally important features. For example, Zheng et al. [23] mined spatial characteristics by devising a global sampling method. They also maximized the extraction of global spatial characteristics by proposing a classification network. However, using solely global characteristics as a basis might produce inaccurate results for categorization. The reason is that if RS images contain the same salient objects, they may be categorized into the same class, despite differences in global features. In this case, relying solely on global features for classification may trigger classification errors [3].
2) Most DNNs need complex structures for AS classification [24]. The training images often have high resolution. To process the images, the DNN methods need a lot of computational resources. They usually rely on platforms with powerful GPUs. The reliance on such hardware underscores the intensive computational resources required by DNNs to effectively perform AS classification. However, deploying DNN models, especially multibranch ones, on embedded or mobile devices is complex and inconvenient. The existing models are usually time-consuming for embedded devices with limited resources [25].

To tackle the first issue, we introduce a feature extraction algorithm for detailed and local feature capture by channel attention. The algorithm leverages three 2-D convolutions followed by a max-pooling operation, aiming to extract initial features and reduce spatial dimensions. Subsequently, we use three convolutions with varying kernel sizes to encompass features across diverse scales. We improve the resilience to alterations in feature positions with (1). Next, the algorithm uses the squeeze function to compress feature dimensions. We use a 1-D convolution operation with dynamically adaptive kernel sizes, which enhances the capability to capture distant correlations and localized details. The unsqueeze function is applied to expand feature dimensions for further processing. We convert attention weight, emphasizing important features while suppressing irrelevant ones based on their attention scores. Then, we downsample the features with the downsample layer in ResNet, facilitating hierarchical feature extraction. The related process is performed iteratively, ensuring comprehensive feature extraction across multiple scales.

Then, we develop a feature refinement algorithm to optimize feature selection. The algorithm begins by applying 2-D convolutions to capture features. We design (6) and solve it to obtain closed-form solutions for the weight of each neuron. Subsequently, we calculate the mean ($\eta_{(y)}$) and variance ($\nu^2_{(y)}$) of activation in each neuron across the spatial dimension. Based on these values, the weight of each neuron is determined using a specific function and further refined using the ReLU function to ensure nonnegativity. Next, the algorithm performs elementwise multiplication to effectively emphasize important features while suppressing less significant ones, resulting in refined features. Finally, the residual structure is adopted to alleviate the vanishing or exploding gradient issue. Overall, the algorithm enhances feature discrimination and robustness by iteratively refining features based on their importance and interaction with the inputs.

To address the second problem, we auto-optimize our two-stage algorithm based on one-shot tuner. The optimization ensures adaptability across various hardware. It involves a cost model that guides the search for suitable hardware backends. We model the relationship between optimized operators and runtime. The computational load is minimized. The process is vital for applications requiring quick response times. The optimization process constructs a compilation dataset from sampled tasks and codes for the training of the cost model. Moreover, by automatically optimizing low-level codes, the optimization method improves performance and reduces the time across different hardware.

C. Contribution

The primary advancements in the proposed approaches are outlined as follows.

1) We construct a feature extraction algorithm to capture interactivity between channels. Unlike the existing methods, the algorithm places greater emphasis on capturing local features effectively. We achieve this through convolutional designs with varying receptive fields to enhance image details and automatic adjustment of convolutional kernel shapes. The algorithm enhances feature aggregation and mines efficient parts of inputs with attention. We emphasize the robust representation of models by exploring local cross-channel interactivity. Our adaptive method of selecting the kernel shape automatically tailors the kernel shape for each specific input. This adaptability ensures that the convolution operations are optimized for inputs.
2) We develop a feature refinement algorithm relying on attention to improve features further. This algorithm enhances the discriminative power of features by assigning adaptive weights. Specifically, we introduce a function to calculate attention weights. The function is different from the existing methods, which often use additional subnetworks to generate attention weights. We assess the importance of a neuron based on its linear separability from others. This innovative approach provides a more detailed understanding of feature importance, leading to more accurate weight assignments than the current technique. Especially, we accentuate crucial features and diminish less relevant ones without additional parameters. This approach avoids excessive

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

structural adjustments and results in a lightweight structure. Experiments on AS classification confirm the effectiveness of the algorithm.

3) We optimize the above two-stage algorithm for AS classification on different devices. Compared with the existing methods, our optimized algorithm boasts three key advantages: adaptability across different devices, high execution efficiency, and effective resource utilization. In detail, we develop highly efficient code using auto-optimization to fit various platforms. The optimization of low-level code is automated based on specific hardware features. This ensures that our methods are seamlessly adaptable and efficient across different hardware configurations. We adopt a cost model derived from the current model. The cost model guides the search for suitable hardware backends. Our method minimizes unnecessary optimization searches by modeling the relationship between optimized operators and their runtime implications. We construct the compiling dataset through sampled tasks and codes to train the cost model. With the trained cost model, optimized codes are obtained, leading to faster deployment. In summary, our research offers a comprehensive solution to the challenges of deploying AS classification on embedded devices.

II. Related Work

Most existing AS classification methods tend to use DNNs, strengthening semantic representation. Thus, we review the DNN-based studies applied in AS classification, that is, basic neural networks, fusion neural networks, attention neural networks, pooling, or loss function improvement networks.

A. Basic Neural Network

For most classification tasks, effective feature representation is crucial. The CNN models emerge as a powerful approach to effectively extract AS features [1], [4], [6], [12], [13], [14], [15], [16], [17], [26]. For example, Lv et al. [1], [4] proposed novel feature extraction methods to process adaptive regions and achieved promising results. The innovation [1] lies in the adaptive region shape similarity (ARSS) approach. ARSS enhances change detection by exploring the context around each pixel, defining direction lines, and comparing histogram curves. The innovation [4] has two novel features. One feature enhances spectral homogeneity around a pixel. The other feature quantifies the connection between the size and configuration of a flexible region. The two features enhance land cover classification accuracy.

Lv et al. [6], Zhang et al. [26] provided a valuable resource for future exploration in RS object detection. They both discussed and reviewed RS image detection, covering literature overviews and classical methods. They guided developments in disaster response and land cover change detection. The study [12] improved parameter and training efficiency through scaling and neural architecture search. He et al. [13] designed residual blocks and avoided exploding gradients in deep CNNs. Relying on the method [13], several models were constructed [14], [15], [16], [17]. DenseNet [14] explored fine feature representation using densely connected convolutions. DPN [15] combined ResNet and DenseNet to learn and share features. RepVGG [16] used structural reparameterization to decouple inference and training. ConvNeXt [17] integrated elements of vision transformers into ResNet [13]. These models [14], [15], [16], [17] all alleviate the vanishing gradient problem. However, these models use fixed convolutional kernel sizes. This can limit their ability to adapt to the varying scales and shapes of features within different images or regions of the same image. To solve this problem, we introduce the automatic adjustment method of changing receptive fields and convolution kernels in this article.

B. Fusion Neural Network

Some researchers fused DNNs with other methods, such as machine learning, for AS classification [3], [9], [27], [28], [29], [30], [31]. Specifically, Xu et al. [3] combined lie group algorithms and CNNs to capture critical information. They enhanced the interpretability of AS classification networks. Li et al. [29] combined key filter bank and DenseNet. They retained global features by capturing regions. A multilevel feature fusion network (MLFF) is developed by [30]. Its innovation lies in the adaptive method to reduce the dimensionality of high-dimensional features and efficiently fuse these features with MLFF. Wang et al. [31] brought the hidden ontological structures to light and recognized multigrained characteristics.

In addition, some authors fused capsule, graph, or adaptive networks with CNNs for AS classification [9], [27], [28]. For example, Xu et al. [9] created graph CNNs. Zhang et al. [27] adopted capsule networks to transform the learned features into several tiers of capsules and output classification information. Wang et al. [28] used adaptive learning to transfer CNNs, finding out which pretrained feature should be transmitted to categorization models. These approaches [3], [9], [27], [28], [29], [30], [31] have demonstrated competitive accuracy in AS classification. However, these fusion strategies bring a computational burden. In contrast, we adopt an automatically optimized method to reduce the computational burden and achieve the adaptability across devices of AS classification in this article.

C. Attention Neural Network

Some researchers introduced attention to CNNs, emphasizing crucial features [5], [8], [21], [22], [24], [32], [33], [34], [35], [36]. Specifically, Tang et al. [5] extracted the contextual features with cross-level attention and multiscale transformer techniques. Tang et al. [21] adopted attention approaches and designed attention-consistent networks to capture important features. Huang et al. [22] also used transformer learn feature relationships and context. Zhao et al. [24] augmented the interaction among channels within the feature map by the attention. For land cover change detection, multiscale dilation and spatial–spectral attention systems are designed to detect land cover change effectively [8]. Zhao et al. [32] increased CNN generalization to extract discriminative information with an attention model. Bi et al. [33] integrated attention poolings into CNNs, enhancing feature depiction.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

They extracted the important local semantics with channel-spatial attention. Zhang et al. [34] addressed the limitations of CNNs in capturing complex temporal–spatial–spectral relationships by designing the cascade-spectral-aware transformer (CAST). Cao et al. [35] brought self-attention and support vector machine techniques into CNNs for classification in AS. Zhang et al. [36] designed the multiscale attention model to mine multiscale and attention features. These existing methods all use attention model to boost the feature extraction performance. Still, they have limitations in capturing the interactivity across channels. To solve such problems, we adopt the cross-channel convolution mechanism without reducing or enlarging dimensions in this article. This helps capture cross-channel dependencies while reducing parameters.

D. Pooling or Loss Function Improvement Network

Some authors introduced poolings and loss functions into CNNs, boosting AS classification accuracy [7], [10], [37], [38], [39]. For instance, Bi et al. [7] considered the categorization of AS to be a multiple-instance learning issue. They solved the issue using instance-level classifiers, poolings, and classification layers. A composite loss function was designed to mitigate unequal sample sizes [10]. He et al. [37] embedded covariance poolings in DNNs. They learned and combined representative features. He et al. [38] integrated multilayer features by covariance poolings. They augmented the feature information of various layers. Xie et al. [39] introduced scale-free CNNs and global average pooling to process features, improving classification accuracy.

Some researchers improved the loss functions for the training of AS classification [40], [41], [42]. For instance, Zhang et al. [40] presented a network for generating features, converting few-shot categorization into standard categorization. They also designed two losses for quality sample generation. Bazi et al. [41] provided a fine-tuning method for deeper models by modifying loss functions. Cheng et al. [42] designed discriminatory CNNs (D-CNNs). They optimized an objective function to train D-CNNs and raise classification performance. However, these improved pooling and loss strategies increase model complexity for AS classification. To address the problem, we develop a new method for assessing the importance of neurons by directly calculating model weights and not relying on additional subnetworks in this article. This approach reduces model complexity based on their linear separability.

III. Methods

This part describes our two-stage algorithm, which contains feature extraction and refinement algorithms. The two-stage algorithm uses cross-channel local interactions, captures important features, and enhances CNN representation.

A. Overall Pipeline

In the first stage, we propose a feature extraction algorithm through improved convolutional layers based on attention. As shown in Fig. 1, we introduce a cross-channel convolution mechanism in the algorithm. The purpose is to boost the accuracy of feature extraction and enhance the robustness of models. Specifically, we first explore the application of 2-D convolution and max-pooling operations to learn and downsample 4-D features. Second, we use convolution kernels of different sizes to capture different levels of feature. This algorithm enhances the model robustness to changes in feature positions through regional average pooling (1). To achieve feature fusion and dimensionality compression, we use compression and expansion methods such as squeeze and unsqueeze functions. Third, we dynamically adjust the size of the convolution kernel with adaptive (3). This method flexibly adapts to different scales of feature representations by dynamically determining the scope of cross-channel interactions within a region. Finally, we introduce the sigmoid function to normalize the weights and use the residual framework to promote the training of deep networks, further boosting the representation ability of features and the depth of CNNs.

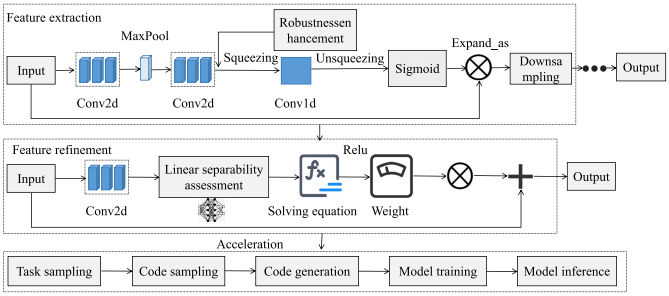


Fig. 1. Overall pipeline.

In the feature extraction algorithm, our contribution lies in emphasizing the interaction between features through a cross-channel convolution mechanism, which appears for the first time in AS classification. In addition, we dynamically adjust the convolution kernel size according to data characteristics. This strategy improves the generalization ability and processing efficiency of models for different data features, thereby leading to our excellent performance in multiple datasets. Third, we improve the depth of the AS classification model by combining 2-D convolution, pooling, convolution kernels of different sizes, feature fusion, and dimensionality reduction techniques, as well as deep network training strategies (such as the introduction of sigmoid functions and residual frameworks). This provides a comprehensive approach to enhance feature representation capabilities of AS classification.

After feature extraction, we further refine and optimize these features. Therefore, we design a feature refinement algorithm in the second stage. This algorithm refines the extracted features through a local and global feature reorganization process. Specifically, the algorithm uses CNN to deeply mine and extract spatial and channel information. In this algorithm, we design an optimization function to calculate the weight of each neuron. We evaluate the local context and global information within each feature channel to recalibrate the original features, ensuring the algorithm focuses on the most critical information. In addition, we adaptively adjust and fuse features of different scales to automatically adjust the focus of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

attention according to the needs of AS classification, thereby capturing local and contextual information more effectively. The newly generated features are combined with the original features through the residual structure to complete the feature refinement process. Experiments demonstrate that these advantages enhance the performance of models on AS classification.

In the feature refinement algorithm, the innovation lies in that we dynamically calculate weights for each neuron based on the linear separability of other neurons within the same channel. This process enhances the discriminative power of the features, improving model accuracy. In addition, the algorithm is highly adaptable to various types of AS data by dynamically adjusting the feature weights and refining features based on actual data statistics. This adaptability ensures that it can maintain high performance even in diverse and challenging scenarios. Unlike other AS classification models, we adopt multiscale feature processing to capture both detailed and abstract features, which is beneficial to understanding both local and global contexts within features.

This article also accelerates AS classification on embedded devices relying on one-shot tuner. Because embedded devices have limited storage and computing resources, we enhance computing efficiency and shorten the inference time of the two-stage algorithm by automatically generating highly optimized codes for them. Specifically, we construct a cost model to guide the search for the hardware backend. This cost model is trained using the collected data so that it can predict the performance of the operator based on its characteristics. The cost model is assessed with validation sets. Continually, the design of the model is iterated based on feedback. The deployable binaries for specific hardware are generated through TVM automatic tuning. The optimized binaries are ready for direct deployment, offering plug-and-play functionality on targeted devices. The result is a leap in inference rates and a reduction in power consumption on embedded devices.

The inference acceleration method offers three advantages over existing AS classification algorithms. First, the approach significantly reduces the computational load by generating optimized operators for the two-stage algorithm. Second, the method uses a cost model to guide the search for optimal hardware backends, enabling the creation of highly optimized binary codes tailored specifically for each embedded device. This contrasts with the current AS classification, which might not account for the unique constraints and capabilities of different hardware environments. Third, the approach decreases the inference time necessary for inference on embedded devices using techniques such as TVM for code generation. Briefly, the acceleration method not only advances the feasibility of performing AS classification in embedded systems but also sets a new benchmark for efficiency and performance.

B. Feature Extraction Algorithm

As shown in Algorithm 1, we extract features and explore local cross-channel interactivity based on [43]. In Algorithm 1, the batch normalization and ReLU operation follows every convolutional layer. In Step 1, we leverage three 2-D convolutions and one MaxPool2d to extract features and perform downsampling, respectively. The kernel shape of the convolutions is 3×3. In Step 2, we also use three 2-D convolutions to extract features again. The kernel shape of the convolutions is, respectively, 1×1, 3×3, and 1×1. In Step 3, we enhance robustness to changes in feature locations with the following equation:

$$y_m = \frac{1}{|\upsilon|} \sum_{\rho \in \upsilon} x_{m_\rho} \tag{1}$$

in the mth feature, y_m is the output value; x_{m_ρ} is the element at point ρ in the region υ; and $|\upsilon|$ corresponds to the total count of elements.

In Step 4, we adopt the squeeze operation to compress feature dimensions, transforming a 4-D feature into a 3-D one. In Step 5, we execute (2) to ensure that all the channels use the same learning settings. The equation indicates the principle of 1-D convolution as follows:

$$O_u = \sum_{p=0}^{I_c - 1} K_e(O_c, p) \star I_n(B_a, p) + \text{bs} \tag{2}$$

where I_n is an input feature; O_u is an output feature; L_e is the length of O_u; a is the index; B_a is the batch shape; O_c and I_c are the count of channels for O_u and I_n, respectively; bs corresponds to a bias element; p is the index of I_c; K_e corresponds to a filter; (B_a, O_c, L_e) corresponds to the shape of O_u; and K_s corresponds to the shape of K_e, which is calculated in an adaptive way

$$k_s = \left| \frac{\log_2(O_c)}{\theta} \right|_o. \tag{3}$$

Here, O_c corresponds to the count of O_c; θ is a coefficient for nonlinear mapping, which is set to 2; and $|(\log_2(O_c)/\theta)|_o$ is the odd number closest to $(\log_2(O_c)/\theta)$. Through the above process, the scope of regional cross-channel interactions is determined [43]. We leverage cross-channel local interactions to improve attention effectiveness and obtain channel weights.

In Step 6, we expand feature dimensions from 3-D into 4-D with the unsqueeze function. The 4-D features are used as the input of the sigmoid function in Step 7. In Step 8, we adopt the expand_as function to increase the attention weights shape until it matches the input shape. In Step 9, we multiply the attention weight tensor by the input tensor. In Step 10, we use the downsample layer in ResNet to downsample features. Steps 2–10 are performed cyclically until the output shape is 8×8 in Step 11. In Step 12, we obtain the feature $\mathbf{D}$ of shape 8×8.

Algorithm 1 provides the feature extraction process. The CNN representation is enhanced through interaction across channels. The interactive coverage is determined by the count of filters. The kernel shape is adaptively selected with (3). This dynamic selection allows our algorithm to learn how to effectively emphasize important features. Thus, the algorithm learns how to focus features effectively. The entire procedure extracts local features, which is highly beneficial for locating small objects. Then, to refine the most important details of features, we design a feature refinement algorithm.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Feature Extraction Algorithm

Require: a 4D input tensor
Ensure: a feature map **D**
1: Perform three 2D convolutions and one MaxPool2d
2: Use three 2D convolutions to extract features
3: Execute (1)
4: Perform the squeeze function to compress feature dimensions and turn the 4D feature into the 3D one.
5: Execute (2)
6: Use the unsqueeze function to expand feature dimensions and turn the 3D feature into the 4D one
7: Convert attention weight with the sigmoid function and obtain attention weight
8: Adjust the shape of attention weight to fit the shape of inputs
9: Each attention weight is multiplied by the corresponding input feature
10: Downsample the features with the downsample layer in ResNet
11: Execute Steps 2–10 until the output shape is 8×8
12: Obtain the feature **D** of shape 8×8

C. Feature Refinement Algorithm

To refine features, we construct Algorithm 2 according to the principle of convolutions and attention [44]. The principle of 2-D convolutions is as follows[1]:

$$s\left(\alpha_\iota, M_{\mathrm{u}_\kappa}\right) = \sum_{\gamma=0}^{E_\iota - 1} e\left(M_{\mathrm{u}_\kappa}, \gamma\right) \star \mathrm{inp}(\alpha_\iota, \gamma) + h\left(M_{\mathrm{u}_\kappa}\right) \tag{4}$$

where the output and input values are, respectively, $s(\alpha_\iota, M_{\mathrm{u}_\kappa})$ and $\mathrm{inp}(\alpha_\iota, \gamma)$; κ is the output channel index; M_{u} and E_ι are the output and input channel numbers, respectively; γ is the input channel index; $\star$ is cross correlation; $h(M_{\mathrm{u}_\kappa})$ is the learnable bias; α_ι refers to the batch size; and $e(M_{\mathrm{u}_\kappa}, \gamma)$ refers to a filter.

The attention principle that this article relies on is

$$\mathbf{D_4} = \mathbf{W} \odot \mathbf{D_3} \tag{5}$$

where $\mathbf{D_3}$ is a feature; and $\mathbf{W}$ is a weight representing its importance for the feature $\mathbf{D_3}$. If the feature is more important, this weight $\mathbf{W}$ is larger. The less important the feature, the smaller the weight $\mathbf{W}$. We offer Algorithm 2 that gives each feature a different weight and refines features. In this way, different features are clearly distinguished. To perform (5), we calculate the feature $\mathbf{D_3}$ and the weight $\mathbf{W}$ in Algorithm 2.

As listed in Step 1 of Algorithm 2, we adopt the feature **D** as the input of (4), outputting the feature $\mathbf{D_1}$. Here, the kernel shape is set to 1×1. In Step 2, we use the feature $\mathbf{D_1}$ as the input of (4), outputting the feature $\mathbf{D_2}$; the kernel shape is set to 3×3. To extract features again, we put $\mathbf{D_2}$ into (4) and produced $\mathbf{D_3}$. Meanwhile, the kernel shape is set to 1×1. Steps 1–3 calculate the feature $\mathbf{D_3}$, namely, the input feature of (5). To calculate the weight $\mathbf{W}$ in (5), we construct

[1]https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

Algorithm 2 Feature Refinement Algorithm

Require: a feature map **D**
Ensure: a new feature map $\mathbf{D_4}$
1: Put the input **D** into (4) and produce $\mathbf{D_1}$
2: Adopt $\mathbf{D_1}$ as the input of (4) and produce $\mathbf{D_2}$
3: Put $\mathbf{D_2}$ into (4) and produce $\mathbf{D_3}$
4: solve (6) and get closed-form solutions $r_y = -\frac{1}{2}(y + \eta_{(y)})f_y$ and $f_y = \frac{-2(\psi_{(y)})}{(\psi_{(y)})^2 + 2\nu^2_{(y)}}$
5: $\eta_{(y)} = \frac{1}{L}\sum_{j=1}^{L} Y_j$
6: $\nu^2_{(y)} = \frac{1}{L}\sum_{j=1}^{L}(Y_j - \eta_{(y)})^2$
7: calculate the weight of neurons $\omega^*_y = 1/2 + \psi^2_{(y)}/4\nu^2_{(y)}$
8: use the ReLU function for ω^*_y, outputting $\mathbf{W}$ and $\mathbf{W} = \mathbf{ReLU}(\omega^*_y)$
9: $\mathbf{D_4} = \mathbf{D_3} \odot \mathbf{W}$
10: $\mathbf{D_5} = \mathbf{D} + \mathbf{D_4}$

a function

$$b_y\left(f_y, r_y, \mathbf{y}, Y_j\right) = \frac{1}{L}\sum_{j=1}^{L}\left(1 + f_y Y_j + r_y\right)^2 + \left(1 - f_y y - r_y\right)^2 \tag{6}$$

where the target neuron is y; others are Y_j in one channel of $\mathbf{D_3}$. The spatial dimension index is j ($j \in (0, L]$). L denotes how many neurons there are on one channel. The bias and weights are, respectively, r_y and f_y. The numbers -1 and 1 are binary labels.

The above equation (6) evaluates the linear separability of a target neuron from others. According to the neuroscience investigation, the importance of a neuron increases with the importance of a feature [44]. The more important a neuron is, the more distinct it is from surrounding neurons and the lower its value $b_y(f_y, r_y, \mathbf{y}, Y_j)$. Therefore, we solve (6) to find the importance of a feature, namely the $\mathbf{W}$ in (5).

As in Step 4 of Algorithm 2, (6) is resolved. We obtain the efficient closed-form solutions $r_y = -(1/2)(y + \eta_{(y)})f_y$ and $f_y = ((-2(\psi_{(y)}))/((\psi_{(y)})^2 + 2\nu^2_{(y)}))$, $\psi_{(y)} = y - \eta_{(y)}$. In Step 5, we calculate the mean $\nu^2_{(y)}$. In Step 6, we calculate the variance $\nu^2_{(y)}$. According to the above results, we obtain the weight of each neuron as in Step 7. Here, we calculate the weight ω^*_y of each neuron by solving (6). These weights are not preset but dynamically calculated based on the statistics of the features. This means that the algorithm adaptively reshapes feature maps by adjusting the importance of each neuron based on the actual performance of the feature.

With the obtained feature $\mathbf{D_3}$ and the weight $\mathbf{W}$, we distinguish features through Step 9 of Algorithm 2. That is, the weight $\mathbf{W}$ is multiplied by the feature $\mathbf{D_3}$, and the feature $\mathbf{D_4}$ is created. This step emphasizes essential features and suppresses insignificant ones. In Step 10, we use the residual structure in ResNet to add $\mathbf{D_4}$ to **D**. The residual structure alleviates the issue of exploding and disappearing gradients while preserving accurate information in model training. Then, the feature refinement process is completed.

Algorithms 1 and 2 are in series and constitute the two-stage algorithm. Algorithm 1 learns more local features and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

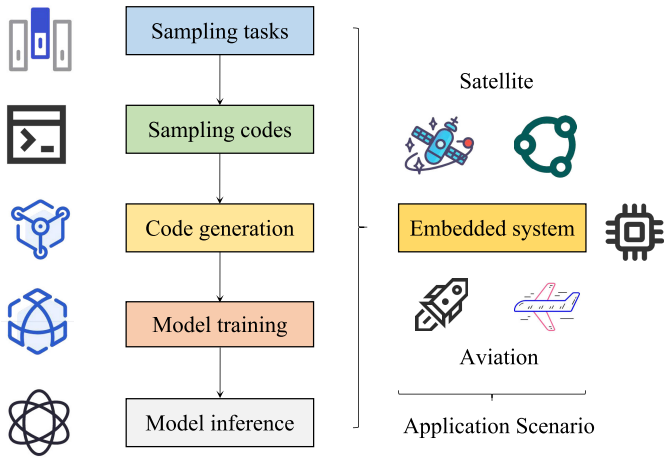


Fig. 2. Inference acceleration method.

cross-channel interaction. Algorithm 2 refines features and selects key information. With the two-stage algorithm, the CNN representation is enhanced. The impact of many small objects is reduced for AS classification. The two-stage algorithm is also conducive to learning features in complex AS. The effect of the two-stage algorithm will be confirmed by the experimental results.

D. Inference Acceleration Method

Based on the two-stage algorithm, we introduce an inference acceleration method [45] to classify AS on embedded devices. Recently, AS classification has generally been based on deep learning (DL). The DL technique necessitates substantial storage and computation. However, existing embedded devices such as drones have a limited amount of computation and power consumption. It is challenging and still in the research stage for embedded devices to classify AS. We provide an optimization technique to increase computing efficiency and shorten the inference time of the two-stage algorithm on embedded devices.

To perform the optimization process, we automatically create highly optimized code to match different embedded devices. First, the model [45] is used to construct the cost model, which directs the search for hardware backends. Through the cost model, the optimized operators of our two-stage algorithm are found. Second, to initialize the cost model, we create a training dataset using efficient sampling tasks and codes, as depicted in Fig. 2. The sampled tasks consist of the convolution and full connection layers in the two-stage algorithm.

Third, we randomly sample tasks and use TVM [46] to perform code generation for each task, resulting in multiple binary codes. Meanwhile, the values of different knobs are also produced. The sampling process is guided by output and input sizes and tensor types. We sample the promising generated code. In code sampling, we choose the value of knobs. In each sampling, a value for each knob is selected. The training label is the inference time. Finally, the cost model is trained and evaluated using the constructed dataset. The optimized operators are preliminarily predicted in model inference. After the training and inference of the cost model, we obtain a pretrained model, which accelerates task learning and decreases the autotuning overhead. After the formal model training and TVM backend, deployable binaries are obtained for specific hardware.

Through the steps in Fig. 2, low-level code optimization is completed for hardware features. The optimization is automatic. We obtain efficient binary codes for each operator of the two-stage algorithm. The binary code is specific for different embedded devices. We also grasp the correlation between optimized operators and their runtime effects. Thus, redundant optimization searches are reduced. The operator optimization for each embedded device leverages the computing power of embedded systems. Furthermore, the inference speed of the two-stage algorithm is accelerated. The effect before and after optimization is evaluated in Section V.

IV. Experiment Description

The datasets, evaluation metrics, and experimental setup are described as follows.

A. Dataset

To evaluate algorithms, we use the OPTIMAL-31 [2], AID [47], NWPU-RESISC45 [48], UC Merced [49], WHU-RS19 [50], and PatternNet [51] datasets.

1) The AID dataset has 30 different classifications. Each classification scene is selected from a variety of international locations. The samples in each classification are diversified. They are annotated by experts in RS image analysis. The complete quantity of AID images amounts to 10 000. The dimensions of images are 600 × 600. The dataset includes images from various remote imaging sensors. It adds a layer of complexity to scene classification tasks compared with single-source datasets like UC Merced. The images are taken across various seasons.
2) The NWPU-RESISC45 dataset has 45 different scene image classifications. Every category contains 700 images. The complete quantity of NWPU-RESISC45 images amounts to 31 500. The dimensions of images are 256 × 256. It possesses significant differences in backdrop, pixel clarity, lighting, object orientation, perspective, and translation. The global coverage of the dataset ensures a comprehensive representation of diverse geographical landscapes. The images were selected to include a wide range of conditions: different weather, seasons, illumination, and imaging conditions.
3) The UC Merced dataset comprises 21 different classifications, encompassing highway, and chaparral, among others. Every category contains 100 images. The complete quantity of UC Merced images amounts to 2100. The dimensions of images are 256 × 256. Several highly overlapping groups, including sparse, medium, and dense residential, are included in the dataset. Because of the overlap, AS categorization faces challenges.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

4) The OPTIMAL-31 dataset comprises 31 categories. Every category has 60 256 × 256 pixel images in it. There are 1860 total images in the dataset. Each image is carefully selected to showcase a range of scenes. The scenes cover runways, meadows, churches, beaches, and airports, among others. This diversity of scenes makes the dataset more challenging and valuable for AS classification.
5) The WHU-RS19 dataset spans 19 scenic categories. It encompasses 950 images. The dimensions of images are 600 × 600. The WHU-RS19 classifications are complex because of differences in clarity and scale. The dataset contains meaningful scenes, such as farmland, meadow, bridge, and river, among others. For each class, there are about 50 samples. This dataset offers researchers a comprehensive resource for developing and evaluating new algorithms in AS classification.
6) The PatternNet dataset encompasses 38 categories. Each category contains 800 images. The dimensions of images are 256 × 256. The PatternNet is sourced through the Google Map API or from Google Earth Visuals. The images are large-scale for RS. The coverage of this dataset is large, including various scenarios. The pixel size varies, with spatial resolutions spanning roughly between 0.062 and 4.693 m/pixel. The high-resolution images, diverse classes, and large-scale coverage make PatternNet valuable for AS classification evaluation, especially in the context of DNNs and other advanced methods.

These datasets are high in diversity and similarity between classes. To ensure an equitable comparison, we select an identical training rate (20%, 50%, and 80%) as used in other studies [5], [27], [31], [35], [39], [41], [42]. These training rates are in line with regularly used training rates. Thus, our results are more easily compared to and validated against earlier studies.

B. Evaluation Metrics

1) *Overall Accuracy (OA):* The average accuracy (AA) closely resembles the OA because each category contains an equal number of images. OA is calculated by dividing the number of correctly identified examples by the total number of examples. It is among the most extensively used assessment measures. The higher the OA, the more effective the algorithm.
2) *Confusion Matrix (CM):* CM assesses the model accuracy. We use the trained model with the highest OA to calculate the CM. The elements of CM are produced through the evaluation sample calculation with estimated and actual classifications.
3) *Inference Time:* The inference time in this article denotes the inference time of algorithms. It reflects the efficiency of algorithms. The shorter the inference time, the higher the efficiency. We assess the inference time of various algorithms on several embedded devices.
4) *Parameters:* The quantity of parameters signifies the magnitude of the models. If a model has a substantial number of parameters, it necessitates greater computational power for both training and prediction. Moreover, the models with more parameters lead to increased energy consumption, posing significant issues for devices reliant on batteries.
5) *Power Consumption:* It is determined by multiplying the current and voltage values. It refers to the amount of electrical energy required by computing hardware to execute a specific algorithm per unit of time. It evaluates the efficiency of an algorithm. Higher power consumption means greater electricity usage per unit of time.

C. Experimental Setup

When training our models on different datasets, we set the same parameters. Our experiment relies on PyTorch and Python 3.6. The operating system is Ubuntu 18.04.5. The batch size is 32. The epoch is 120. The optimizer during training is "torch.optim.SGD" in PyTorch. The preliminary learning rate is 10^{-2}. Our function to adjust learning rates is "StepLR" in PyTorch. The step size and multiplication factor for updating learning rates are 10 and 0.9, respectively. To lessen the impact of randomness on the outcomes, we perform the same experiment five times and calculated the mean and standard deviations of accuracy and time cost following the studies [28], [32]. The standard deviation is the square root of the variance. It is also a measure of the spread of results. The experiments [28], [32] are also conducted five times to determine the mean and standard deviation of the accuracy measurements. The same number of experiments and evaluation metrics help in comparing our approach with current methods.

Various methods are selected for comparative evaluation. These methods include classic and recent research. Their experiment parameters are displayed in Table I. The experimental parameters enhance the understanding of the variables affecting AS categorization. The systematic presentation is pivotal for discerning the impacts of distinct experimental setups on AS categorization results. Each method adopts different libraries such as PyTorch, TensorFlow, and MATLAB. These methods use a range of optimizers, including Adam, SGD, momentum, NAG, RMSprop, and Adagrad. The learning rates range from 10^{-8} to 10^{-2}, with momentum typically set at 0.9. Weight decay values, where applied, vary from 5×10^{-6} to 10^{-2}, indicating regularization intensity. This comprehensive summary shows the computational preferences in remote sensing applications.

V. Results

This section showcases the outcomes on three datasets and on different devices.

A. Overall Accuracy

Table II looks into the impact of changing learning rate parameters on the AA of our model. The table focuses on two primary learning rate parameters: step size and multiplication factor. The step size refers to how frequently the learning rate is adjusted. The multiplication factor represents the factor by which the learning rate is multiplied at each step. The WHU-RS19 dataset is adopted. The training ratio is 50%.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXPERIMENTAL PARAMETERS

Method	Library	Optimizer	Learning rate	Momentum	Weight decay
LGRIN [3]	Pytorch	Adam	10^{-8}	0.9	5×10^{-4}
EMTCAL [5]	Pytorch	Adam	3×10^{-5}	0.9	10^{-2}
MIDC-Net_CS [7]	TensorFlow	Adam	10^{-3}	0.9	5×10^{-6}
DFAGCN [9]	Pytorch	Adam	10^{-3}	0.9	0
ACNet [21]	Pytorch	Adam	10^{-3}	0.9	5×10^{-3}
Inception-v3-CapsNet [27]	TensorFlow	Adam	10^{-3}	0.9	5×10^{-6}
T-CNN [28]	PyTorch	Adam	10^{-3}	0.9	0
KFBNet [29]	Pytorch	Adam	10^{-4}	0.9	0
ACR-MLFF [30]	PyTorch	Adam	3×10^{-4}	0.9	10^{-5}
MG-CAP [31]	TensorFlow	Momentum	10^{-3}	0.9	5×10^{-4}
ResNet-50+EAM [32]	Mxnet	NAG	10^{-3}	0.9	5×10^{-4}
CNN-MS2AP [33]	PyTorch	Adam	10^{-4}	0.9	0
SAFF [35]	PyTorch	Adam	10^{-3}	0.9	0
MSA-Network [36]	Tensorflow	Adam	10^{-4}	0.9	5×10^{-6}
SCCov [37]	MATLAB	Adagrad	10^{-3}	0.9	5×10^{-4}
MSCP [38]	MATLAB	SGD	10^{-3}	0.9	5×10^{-4}
SF-CNN [39]	Pytorch	SGD	10^{-2}	0.9	10^{-4}
EfficientNet-B3-aux [41]	PyTorch	RMSprop	10^{-4}	0.9	0
D-CNN [42]	PyTorch	Adam	10^{-3}	0.9	0
CCNet [52]	Pytorch	SGD	10^{-2}	0.9	5×10^{-4}
CSCANet [53]	Pytorch	SGD	6×10^{-3}	0.9	5×10^{-4}
Ours	Pytorch	SGD	10^{-2}	0.9	10^{-4}

TABLE II
IMPACT OF UPDATING LEARNING RATE PARAMETERS ON AA

Step size	Multiplication factor	Average accuracy
10	0.9	$\mathbf{96.16 \pm 0.81}$
10	0.8	96.02 ± 0.91
10	0.7	96.06 ± 1.70
20	0.9	95.68 ± 0.62
20	0.8	93.76 ± 5.12
20	0.7	95.24 ± 2.87
30	0.9	94.60 ± 2.03
30	0.8	95.56 ± 0.78
30	0.7	95.92 ± 1.40
40	0.9	95.68 ± 1.03
40	0.8	95.16 ± 0.97
40	0.7	96.04 ± 1.17

TABLE III
CONTRIBUTION OF EACH ALGORITHM TO RESULTS

Algorithm 1	Algorithm 2	Accuracy	Time cost
	✓	93.64 ± 1.50	5.69 ± 0.02
✓		95.92 ± 1.62	6.26 ± 0.02
✓	✓	$\mathbf{96.16 \pm 0.81}$	6.34 ± 0.03
✓	✓	$\mathbf{96.16 \pm 0.81}$	$\mathbf{2.70 \pm 0.01}$

The accuracy values are presented as "mean accuracy ± standard deviation," indicating both the average performance and variability in performance. With a step size of 10, the results seem to be relatively consistent across different multiplication factors. The accuracy is highest with a multiplication factor of 0.9 and slightly decreases as the factor is reduced. With a step size of 20, the most variability is displayed. Interestingly, the multiplication factor of 0.8 results in a significantly lower AA and a much higher standard deviation, indicating unstable or inconsistent results. With a step size of 30, the accuracy is quite balanced across the multiplication factors, with the highest accuracy achieved at a factor of 0.7. When the step size is 40, the results are relatively consistent. The multiplication factor of 0.9 generally provides higher accuracies with a smaller standard deviation for most step sizes. The multiplication factor of 0.8 shows the most variability, especially at a step size of 20. The multiplication factor of 0.7 works best with larger step sizes (30 and 40). In summary, the table highlights the nuances of adjusting learning rate parameters on model accuracy. The step size of 10 and multiplication factor of 0.9 provide significant improvements in accuracy. All our subsequent results are based on the step size 10 and multiplication factor 0.9.

Table III investigates how Algorithms 1 and 2 contribute to experimental results. The checkmark (✓) indicates which algorithms are used to produce results. The WHU-RS19 dataset is used. The training proportion is 50%. The time cost is detailed in the final column. The last row shows the accelerated results of our algorithm. When only Algorithm 1 is used, the model achieves an AA of 95.92%. This suggests that Algorithm 1 is effective. When only Algorithm 2 is adopted, the AA achieves 93.64%. It is slightly less effective than Algorithm 1 when used individually.

When both Algorithms 1 and 2 are combined, the AA increases slightly to 96.16%, and the processing time is 6.34 ms. The combination reduces variability (as indicated by the smaller standard deviation) compared with using Algorithms 1 or 2 individually, suggesting that the combined

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

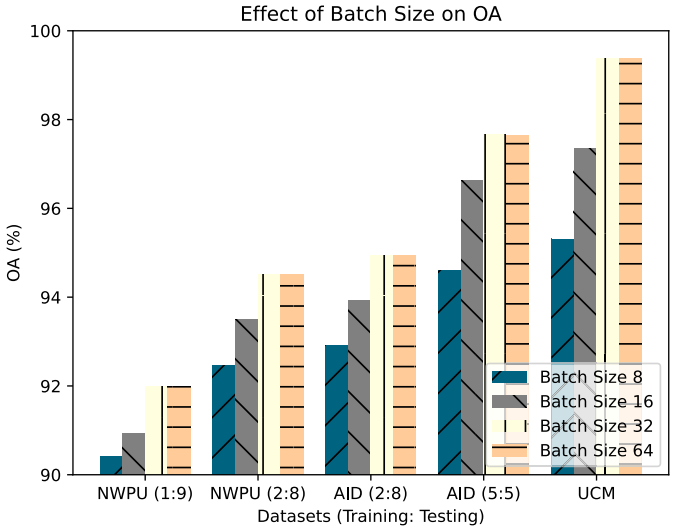


Fig. 3. Effect of batch size on OA.

approach is more stable. The combined approach has a small effect on time cost, making it slightly longer than using Algorithm 1 alone. Where both the algorithms are accelerated together in the last row, the time is significantly reduced to 2.70 ms. This is the shortest time among all the configurations. In summary, using both the algorithms together yields the highest AA. The acceleration is effective. This suggests that there are complementary strengths between the two algorithms that enhance performance when combined. Thus, all our following outcomes derive from the combination of Algorithms 1 and 2. The comparison in Figs. 3–6 also demonstrates that our proposed approach is faster compared with others.

Fig. 3 illustrates the OA of our algorithm across different datasets with varying batch sizes. Five datasets are selected: NWPU (1:9), NWPU (2:8), AID (2:8), AID (5:5), and UCM. The numbers in parentheses indicate the proportion of the training set compared with the test set. The OA percentage increases along the vertical axis, starting at 90%. The batch sizes are adjusted to examine their impact. Four common batch sizes are compared: 8, 16, 32, and 64. Each batch size is represented by different patterns on the bars in the figure, such as diagonal lines, crosshatching, vertical lines, and solid fill, respectively. The figure has a legend at the bottom right. The OA percentages range from 90% to above 99%, with the general trend showing higher accuracy with larger batch sizes. Using the AID (2:8) dataset as a reference, the OA increases from 92.91% to 93.92% when the batch size changes from 8 to 16. However, with the batch size set at 32, our OAs largely stay the same as those observed with the batch size of 64. Hence, we consider the 32 batch size for experiments on different remote sensing image datasets.

We contrast the OA of different methods to demonstrate our effectiveness in the same training environment. Table IV displays the final results on AID. No matter whether the training proportion is 20% or 50%, the two-stage algorithm achieves competitive results. The OA of our method attains 94.94% at the 20% training ratio, outperforming others. The OA of our method reaches 97.16% at 50% training ratio. We show

TABLE IV
AA AND STANDARD DEVIATION COMPARISON ON AID

Method	Training ratio (20%)	Training ratio (50%)
LGRIN [3]	94.74±0.23	97.65±0.25
EMTCAL [5]	94.69±0.14	96.41±0.23
MIDC-Net_CS [7]	88.51±0.41	92.95±0.17
DFAGCN [9]	-	94.88±0.22
ACNet [21]	93.33±0.29	95.38±0.29
Inception-v3-CapsNet [27]	93.79±0.13	96.32±0.12
T-CNN [28]	94.55±0.27	96.72±0.23
KFBNet [29]	94.27±0.02	97.19±0.07
ACR-MLFF [30]	92.73±0.12	95.06±0.33
MG-CAP [31]	93.34±0.18	96.12±0.12
ResNet-50+EAM [32]	93.64±0.25	96.62±0.13
CNN-MS2AP [33]	92.19±0.22	94.82±0.20
SAFF [35]	90.25±0.29	93.83±0.28
MSA-Network [36]	93.53±0.21	96.01±0.43
SCCov [37]	93.12±0.25	96.10±0.16
MSCP [38]	92.21±0.17	95.56±0.18
SF-CNN [39]	93.60±0.12	96.66±0.11
EfficientNet-B3-aux [41]	94.19±0.15	96.56±0.14
D-CNN [42]	90.82±0.16	96.89±0.10
CCNet [52]	92.50±0.46	95.34±0.16
CSCANet [53]	94.67±0.20	96.83±0.14
Ours	$\mathbf{94.94 \pm 0.12}$	$\mathbf{97.66 \pm 0.09}$

TABLE V
AA AND STANDARD DEVIATION COMPARISON ON NWPU-RESISC45

Method	Training ratio (10%)	Training ratio (20%)
LGRIN [3]	91.91±0.15	94.43±0.16
EMTCAL [5]	91.63±0.19	93.65±0.12
MIDC-Net_CS [7]	86.12±0.29	87.99±0.18
DFAGCN [9]	-	89.29±0.28
ACNet [21]	91.09±0.13	92.42±0.16
Inception-v3-CapsNet [27]	89.03±0.21	92.60±0.11
T-CNN [28]	90.25±0.14	93.05±0.12
KFBNet [29]	90.27±0.02	92.54±0.03
ACR-MLFF [30]	90.01±0.33	92.45±0.20
MG-CAP [31]	90.83±0.12	92.95±0.13
ResNet-50+EAM [32]	90.87±0.15	93.51±0.12
CNN-MS2AP [33]	87.91±0.19	90.98±0.21
SAFF [35]	84.38±0.19	87.86±0.14
MSA-Network [36]	90.38±0.17	93.52±0.21
SCCov [37]	89.30±0.35	92.10±0.25
MSCP [38]	88.07±0.18	90.81±0.13
SF-CNN [39]	89.89±0.16	92.55±0.14
EfficientNet-B3-aux [41]	91.08±0.14	93.81±0.07
D-CNN [42]	89.22±0.50	91.89±0.22
CCNet [52]	87.74±0.19	91.18±0.26
CSCANet [53]	91.27±0.11	93.72±0.10
Ours	$\mathbf{91.99 \pm 0.10}$	$\mathbf{94.52 \pm 0.08}$

relatively low variability, indicating consistent performance. The two-stage algorithm is 0.21% better than the research [3], 0.26% higher than the EMTCAL model [5], 0.71% better than the classical KFBNet [29], 2.64% higher than CCNet [52], 0.29% better than the latest model CSCANet [53], and 0.41% higher than others at the 20% training ratio. The two-stage algorithm also surpasses the comparison literature at the 50% training ratio. Our method stands out as the best-performing method in a comprehensive comparison on AID. The results verify the superiority of our two-stage algorithm.

Table V describes the outcomes on NWPU-RESISC45. Our method exhibits impressive performance, especially at the 20% training ratio, where it is among the top performers. Our algorithm attains 91.99% in OA if the training proportion

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

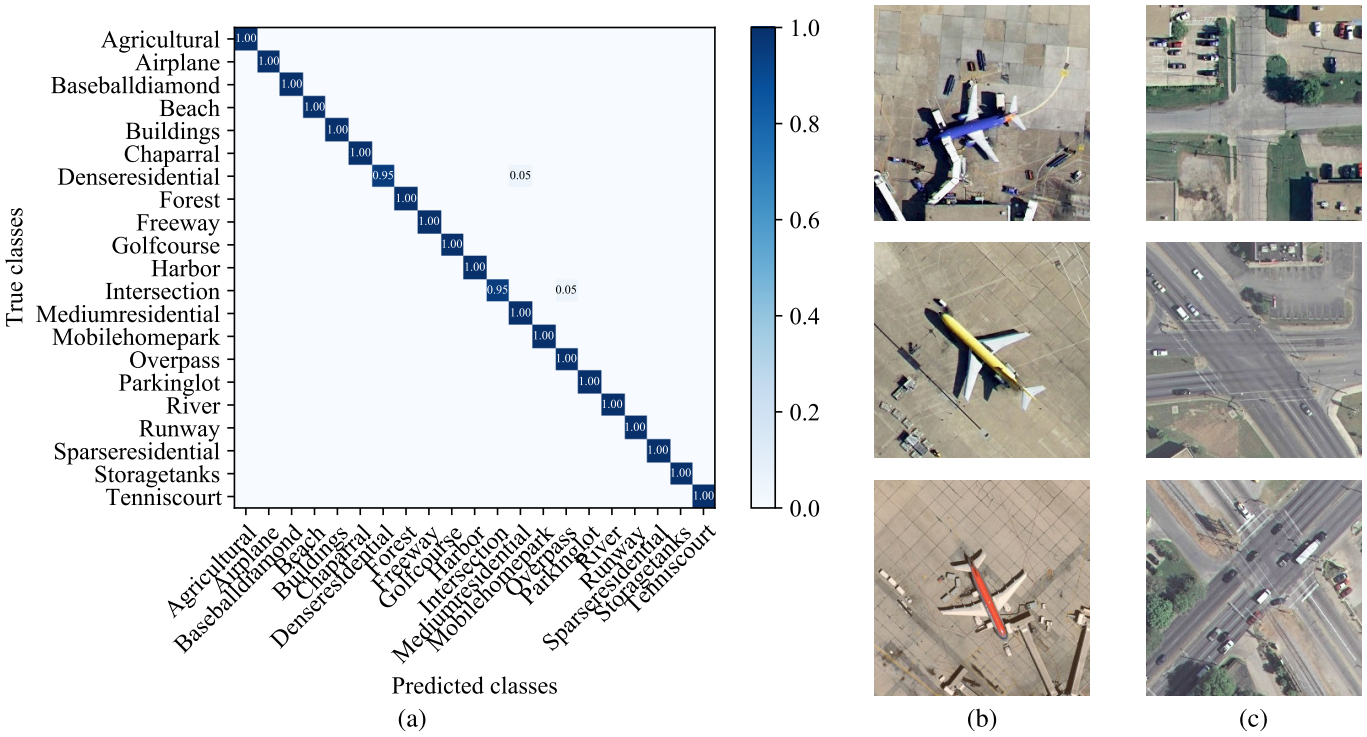


Fig. 4. (a) CM of the UCM dataset at training ratio 80%. (b) Airplane. (c) Intersection.

TABLE VI
AA AND STANDARD DEVIATION COMPARISON ON UC MERCED

Method	Training ratio (80%)
LGRIN [3]	98.97±0.31
MIDC-Net_CS [7]	97.40±0.48
DFAGCN [9]	98.48±0.42
Inception-v3-CapsNet [27]	99.05±0.24
T-CNN [28]	99.33±0.11
ACR-MLFF [30]	99.37±0.15
MG-CAP [31]	99.00±0.10
ResNet-50+EAM [32]	98.98±0.37
CNN-MS2AP [33]	99.01±0.42
SAFF [35]	97.02±0.78
MSA-Network [36]	98.96±0.21
SCCov [37]	99.05±0.25
MSCP [38]	98.40±0.34
SF-CNN [39]	99.05±0.27
EfficientNet-B3-aux [41]	99.09±0.17
D-CNN [42]	98.93±0.10
CCNet [52]	99.11±0.30
CSCANet [53]	**99.76 ± 0.16**
Ours	99.38±0.11

accounts for 10%, outperforming the latest research [53] by 0.79%, the recent research [3] by 0.08%, the EMTCAL model [5] by 0.39%, the classical KFBNet [29] by 1.91%, and others at least by 0.99%. When the training proportion is 20% in OA, the two-stage algorithm attains 94.52%, exceeding the latest research [53] by 0.85%, the recent research [3] by 0.09%, the EMTCAL model [5] by 0.93%, the classical KFBNet [29] by 2.14%, and others at least by 0.76%. The comparison confirms that the two-stage algorithm enhances the capacity for AS categorization. In summary, the table offers a detailed comparison of different models on NWPU-RESISC45. Our method stands out as one of the top contenders.

Table VI assesses OA on UC Merced. The performance is evaluated based on an 80% training ratio. The performance of the two-stage algorithm does not reach the accuracy levels achieved by the CSCANet, which we identify as an improved direction for our future research. The "CSCANet" and our method are among the top-performing methods, both achieving an accuracy of over 99.3%. Particularly, our two-stage algorithm is one of the best performers for this dataset and training ratio. For example, the two-stage algorithm attains 99.38% in OA, exceeding the LGRIN [3] by 0.41%, the ACR-MLFF [30] by 0.01%, the recent T-CNN [28] by 0.05%, and others at least by 0.33%. The comparison result shows that our superiority has been reflected when compared with similar approaches. The two-stage algorithm leads to improved feature representation in complex AS on UC Merced. The feature representation is essential to improving the OA.

Table VII showcases the impact of different sample training rates on the AA. In this table, six different datasets are adopted. For each dataset, there are three distinct training ratios: 20%, 50%, and 80%. The goal is to show how changing the amount of training data impacts AA on different datasets. Overall, when the proportion of training grows (ranging from 20% to 80%), there is a general trend of increased accuracy across all the datasets. This suggests that providing more data for training generally improves the model performance. The UC Merced dataset has the most significant relative improvement in accuracy, from 92.38% to 99.38% when the proportion of training grows from 20% to 80%. Some datasets, such as "PatternNet," start with extremely high accuracy, even at the 20% training ratio. Some datasets like "AID" and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
IMPACT OF DIFFERENT SAMPLE TRAINING RATES ON AA

Dataset	Training ratio (20%)	Training ratio (50%)	Training ratio (80%)
OPTIMAL-31	85.75 ± 0.47	93.33 ± 0.23	93.93 ± 0.15
UC Merced	92.38 ± 0.69	97.74 ± 1.17	99.38 ± 0.11
PatternNet	99.56 ± 0.03	99.74 ± 0.02	$\mathbf{99.80 \pm 0.01}$
AID	94.94 ± 0.12	97.66 ± 0.09	97.77 ± 0.02
NWPU-RESISC45	94.52 ± 0.08	96.38 ± 0.04	97.25 ± 0.15
WHU-RS19	94.84 ± 1.19	96.16 ± 0.81	98.69 ± 0.61

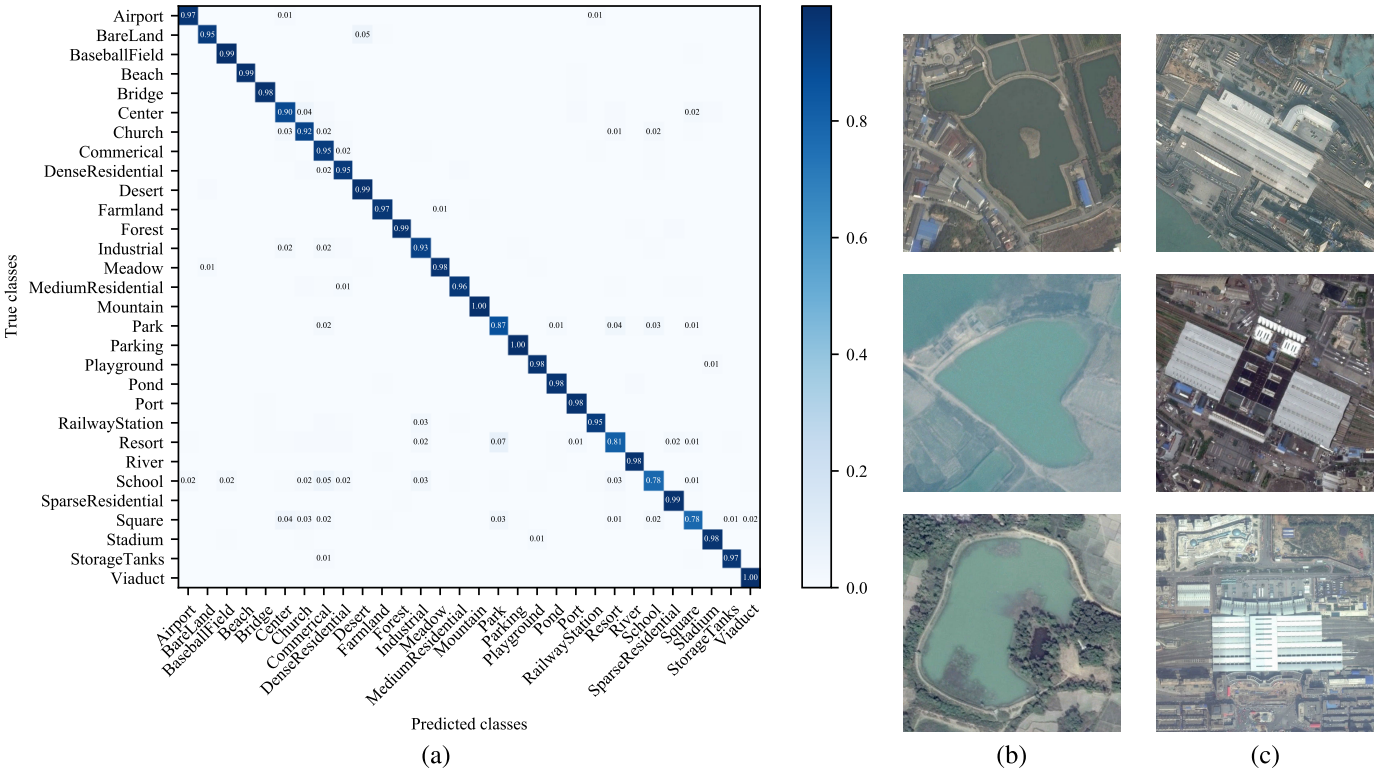


Fig. 5. (a) CM of the AID dataset at training ratio 20%. (b) Pond. (c) Railway Station.

"PatternNet" show minimal improvement when increasing the training ratio from 50% to 80%. In summary, the table highlights the importance of training data quantity and datasets for AA. Most results benefit from an increased training ratio, with accuracy generally improving as more data are used for training. Therefore, we train and assess models with the same ratios. We also depict the CM to convey the superiority of the two-stage algorithm as follows.

B. Confusion Matrix

The CM has become an essential metric in scene classification. Fig. 4(a) depicts the CM of our two-stage algorithm on the UCM dataset. The horizontal and vertical axes show the predicted and true categories. Fig. 4(b) and (c) shows the samples from the airplane and intersection classes. Our two-stage algorithm is accurate for most scene classifications. For example, the accuracy of some classification results reaches 100%, like baseball diamond, agricultural, airplane, beach, buildings, and chaparral, among others. Classification accuracy reaches 100% for 19 of the 21 classifications. Only the classification accuracy of dense residential and intersection is 95%. The intersection and dense residential images contain more buildings. The building structure and texture composition are complex. Addressing the challenge is our next major work.

Fig. 5(a) depicts the CM of the two-stage algorithm on AID. Fig. 5(b) and (c) displays classes like pond and railway station. The accuracy of our two-stage algorithm is again demonstrated. For example, in 26 out of the 30 categories, the two-stage algorithm achieves a classification accuracy exceeding 90%. The accuracy of some categories reaches 100%, like those of a mountain, parking, or viaduct. The classification accuracy of a park or resort is over 80%. The classification accuracy of school and square is not very high due to the poor texture information. In most categories, the two-stage algorithm attains high classification accuracy.

The CM of the two-stage algorithm is illustrated in Fig. 6(a). The encouraging outcomes on NWPU-RESISC45 demonstrate our potency. Fig. 6(b) and (c) depicts the examples from the railway and forest classes. Specifically, the accuracy of the cloud scene reaches 100%. In 42 out of the 45 categories, our classification accuracy exceeds 90%. Only the wetlands and churches have classification accuracy less than 90%. The

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

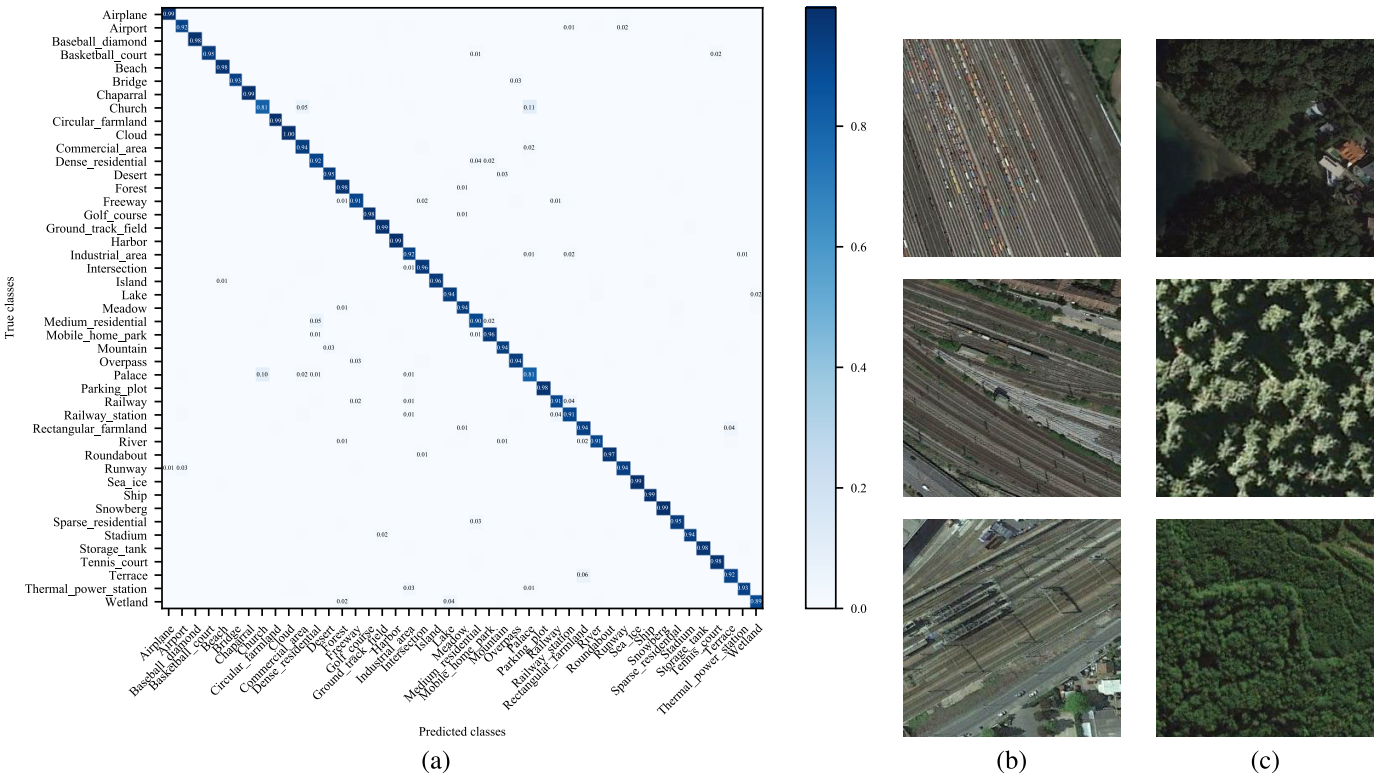


Fig. 6. (a) CM of the NWPU-RESISC45 dataset at training ratio 20%. (b) Railway. (c) Forest.

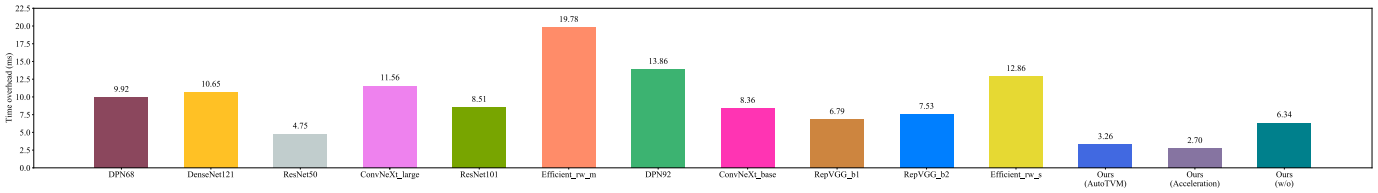


Fig. 7. Inference time on RTX 2080Ti GPU.

reason is that the texture composition of these structures is complex. Our next important task is to deal with the challenge. Our classification accuracy is precise for the majority of scene categories. The reason is that the two-stage algorithm has fine feature representation. Beyond the contrasts, we evaluate the inference time of the two-stage algorithm.

C. Inference Time

To confirm our efficiency, we count the inference time of comparison methods. The input shape is 256 × 256. In the comparison approaches, "efficient_rw_m" and "efficient_rw_s" denote "EfficientNetV2_rw_m" and "EfficientNetV2_rw_s" in the literature [12], respectively. "Ours (AutoTVM)" represents the two-stage algorithm optimized by AutoTVM [46]. "Ours (w/o)" stands for the two-stage algorithm without acceleration. "Ours (Acceleration)" represents the two-stage algorithm optimized by our inference acceleration method.

The inference time on the inference device (RTX 2080Ti GPU) is displayed in Fig. 7. The two-stage algorithm has the lowest time expenses. For example, our two-stage algorithm takes 67.95% less time than the "EfficientNetV2_rw_m" [12]. We accelerate the inference of the two-stage algorithm. Our inference time is reduced from 6.34 to 2.70 ms. Our accelerated algorithm has 43.16% and 68.27% less time than the famous ResNet50 and ResNet101 [13], respectively. Compared with DenseNet121 [14], our accelerated algorithm has 74.65% shorter time. We reduce the time expense by 72.78% and 80.52% than DPN68 and DPN92 [15], respectively. We conserve the inference time by 60.24% and 64.14% than the lightweight RepVGG_b1 and RepVGG_b2 [16], respectively. We also decrease the inference time by 67.70% and 76.64% than ConvNeXt_base and ConvNeXt_large [17], respectively. Compared with "Ours (AutoTVM)," our accelerated algorithm reduces the inference time by 17.18%. The low inference time of our approach facilitates its practicability in AS.

Another inference device (Titan V GPU) is used to verify the superiority of our methods. On this device, the inference time of the original two-stage algorithm is 9.17 ms in Fig. 8, which is shorter than most comparison methods [12], [14], [15], [16], [17]. Our accelerated algorithm is 65.32% faster than before. Compared with AutoTVM, our acceleration is 10.67% faster. Our accelerated algorithm has 90.13%

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

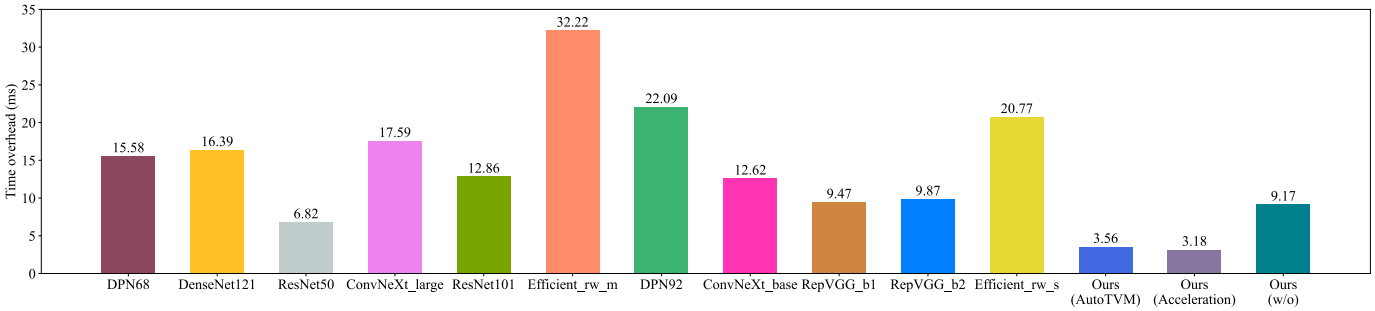


Fig. 8. Inference time on Titan V GPU.

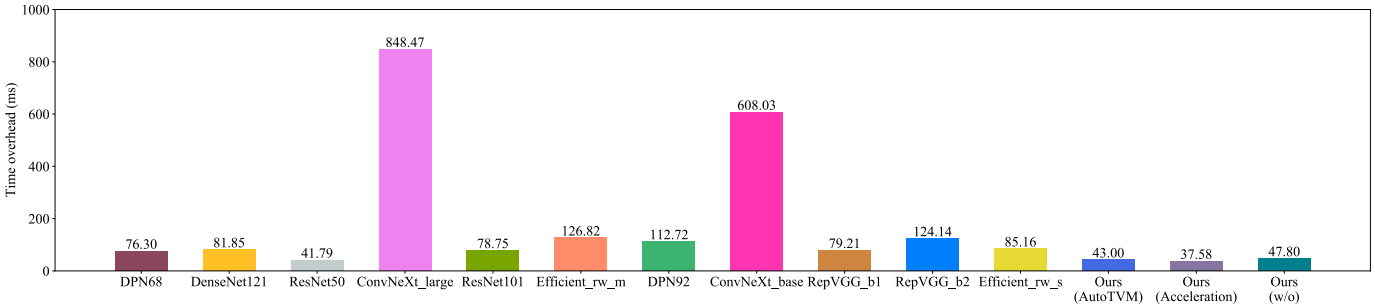


Fig. 9. Inference time on Jetson TX2.

and 84.69% shorter time than the lightweight "EfficientNetV2_rw_m" and "EfficientNetV2_rw_s" [12], respectively. The accelerated algorithm reduced 53.37% and 66.42% less time than the classic ResNet50 and RepVGG_b1, respectively. Compared with DPN68 and DenseNet121, we improve the speed by 3.90× and 4.15×, respectively. Compared with ConvNeXt_base and ConvNeXt_large, we reduce the inference time by 74.80% and 81.92%, respectively. In summary, our accelerated algorithm has the lowest inference time on Titan V GPU.

To assess the inference time on Jetson TX2, we compare the two-stage algorithm with recent studies in Fig. 9. The promising outcomes reflect our efficiency. For example, the latency of the two-stage algorithm is 47.80 ms. Our accelerated algorithm is 0.27× faster than before. Compared with AutoTVM, our accelerated method attains 0.14× speedup. The accelerated algorithm consumes 70.37% and 55.87% less time than "EfficientNetV2_rw_m" and "EfficientNetV2_rw_s," respectively. When compared with ResNet101 and RepVGG_b2, our accelerated algorithm is 52.28% and 69.73% faster, respectively. We decrease the inference time by 54.09% and 66.67% than DenseNet121 and DPN92, respectively. We also lessen the inference time by 95.57% and 93.82% than ConvNeXt_large and ConvNeXt_base, respectively. These results fully demonstrate the effective acceleration of the two-stage algorithm. These observations clearly validate our speed-up capability.

The inference time of different approaches on Jetson NX is shown in Fig. 10. The inference time of the two-stage algorithm is 37.02 ms, which is 66.47% lower than the representative "EfficientNetV2_rw_m." The inference time of our accelerated algorithm is 22.84 ms, which is lower than 23.43 ms of the AutoTVM approach. When compared with "EfficientNetV2_rw_s," our accelerated algorithm is 70.68% faster. Compared with RepVGG_b1 and RepVGG_b2, our accelerated algorithm consumes 62.77% and 75.78% less time, respectively. We decrease the inference time by 74.87% and 55.56% than DPN92 and DPN68 [15], respectively. Compared with the latest research ConvNeXt_base and ConvNeXt_large, our accelerated algorithm takes 95.79% and 96.79% shorter time. These outcomes confirm our capacity to accelerate AS classification.

The inference time of different algorithms on Jetson Nano is recorded in Fig. 11. We greatly decrease the inference time, leading to fast inference on embedded devices with limited resources. For example, the time cost of the original two-stage algorithm is at least 14.34% less than that of the recent methods [12], [14], [15], [16], [17]. Our acceleration method is 9.7% faster than AutoTVM. Compared with "EfficientNetV2_rw_m," our accelerated algorithm has 68.03% lower time. Our accelerated algorithm has 50.75% and 67.23% less inference time than RepVGG_b1 and RepVGG_b2, respectively. Compared with DPN92 and DPN68, our accelerated algorithm achieves 62.28% and 19.80% speedup, respectively. Compared with ResNet50 and ResNet101, our accelerated algorithm is 8.08% and 47.76% faster, respectively. The accelerated algorithm also has 85.62% and 91.77% lower time than ConvNeXt_base and ConvNeXt_large, respectively. We reduce the inference time of scene classification in AS effectively on different platforms. In sum, we accelerate the progress and implementation of AS classification on distinct hardware platforms.

D. Parameters and Power Consumption

We select representative methods for the comparison of parameters and power consumption in Table VIII. The quantity of parameters signifies the model size, which remains

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

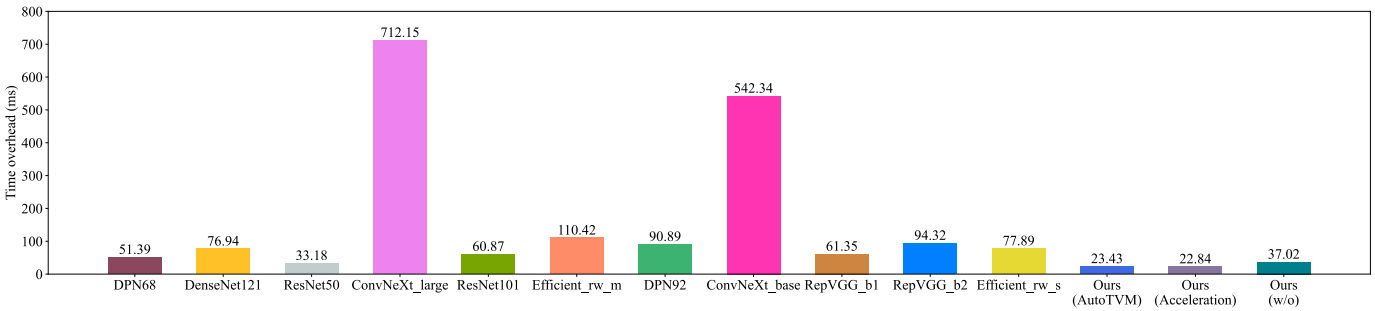


Fig. 10. Inference time on Jetson NX.

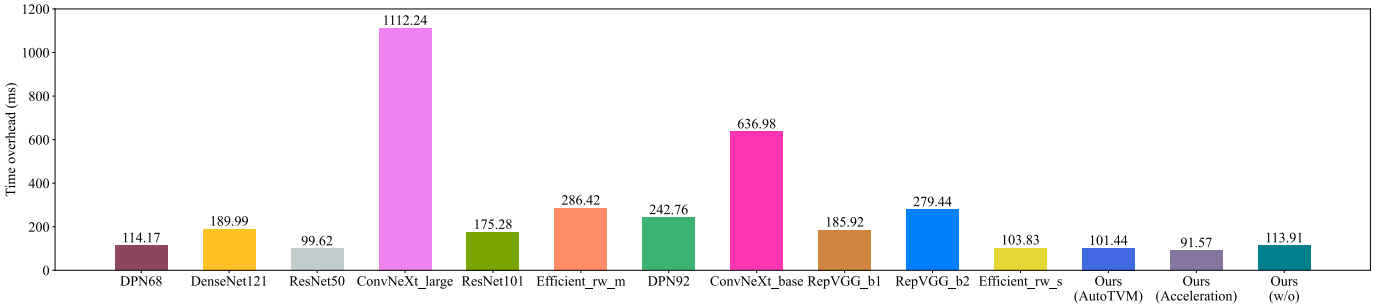


Fig. 11. Inference time on Jetson Nano.

TABLE VIII
PARAMETERS AND POWER CONSUMPTION

Method	Parameters	Power consumption
Inception V3 [3]	45	7.2
ResNet50 [13]	26	7.1
ViT-B/32 [18]	151	8.4
ViT-L/14 [18]	437	9.9
D-CNN [19]	138	8.2
GLANet [19]	138	8.3
ViT-L/16 [22]	304	9.2
ViT-B/16 [22]	87	7.9
AlexNet [22]	61	77
ESD-MBENet-v2-E [24]	24	6.7
VGG-VD-16 [47]	138	8.1
CaffeNet [47]	61	7.6
Ours	**23**	**6.3**

unaffected by variations in the input size. We record the current and voltage of all the models on the NVIDIA TX2 device in Max-N mode using a voltmeter and an ammeter, respectively. We use an image from UC Merced in the current and voltage detection process. The input size is 256 $\times$ 256. We compute the power consumption of all the models using the product of current and voltage. The proposed method ("our algorithm without acceleration") shows improvements in both parameters and power consumption compared with others. We consume the least power at 6.3 watts, standing out for its energy efficiency. It is also characterized by having a minimal number of parameters, totaling 23 million. These results showcase its highly efficient design. Relative to the typical Inception V3 and ResNet50, our parameters are, respectively, 48.9% and 11.5% lower, and our power consumption is, respectively, 12.5% and 11.3% reduced. In addition, compared with ViT-B/16, ViT-L/14, ViT-B/32, ViT-L/16, and our method, respectively, decrease parameters by 73.56%, 94.74%, 84.77%, and 92.43%; we, respectively, reduce 25%, 36.36%, 31.52%, and 20.25% power consumption. This comparison showcases our efficiency in minimizing resource usage while maintaining effectiveness.

VI. CONCLUSION

In this article, we offer enhanced feature extraction and refinement algorithms for AS categorization. In the first algorithm, we capture features and local cross-channel interactivity by designing a feature extraction algorithm. This part aggregates information and investigates several tiers of features. In the second algorithm, we develop a feature refinement algorithm to distinguish compelling features. This part takes the output of the first part as input. It calculates attention weights automatically, without any parameters. In this manner, we converge on key areas and directly output classification results. As demonstrated by the promising outcomes on three datasets, the two-stage algorithm captures the interactivity between channels more effectively and enhances feature discriminative power by assigning adaptive weights. In addition, we introduce an acceleration method to optimize the two-stage algorithm. The acceleration method produces lightweight codes for the two-stage algorithm. Without raising the computing cost, our time spent on different embedded devices is reduced. Future studies will concentrate on further improvement in the accuracy of AS categorization.

REFERENCES

[1] Z. Lv, F. Wang, W. Sun, Z. You, N. Falco, and J. A. Benediktsson, "Landslide inventory mapping on VHR images via adaptive region shape similarity," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–11, 2022, Art. no. 5630211.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

[2] Q. Wang, S. Liu, J. Chanussot, and X. Li, "Scene classification with recurrent attention of VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1155–1167, Feb. 2019.

[3] C. Xu, G. Zhu, and J. Shu, "A lightweight and robust lie group-convolutional neural networks joint representation for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022, Art. no. 5501415.

[4] Z. Lv, P. Zhang, W. Sun, J. A. Benediktsson, J. Li, and W. Wang, "Novel adaptive region spectral–spatial features for land cover classification with high spatial resolution remotely sensed imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5609412.

[5] X. Tang, M. Li, J. Ma, X. Zhang, F. Liu, and L. Jiao, "EMTCAL: Efficient multiscale transformer and cross-level attention learning for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5626915.

[6] Z. Lv et al., "Land cover change detection with heterogeneous remote sensing images: Review, progress, and perspective," *Proc. IEEE*, vol. 110, no. 12, pp. 1976–1991, Dec. 2022.

[7] Q. Bi, K. Qin, Z. Li, H. Zhang, K. Xu, and G.-S. Xia, "A multiple-instance densely-connected ConvNet for aerial scene classification," *IEEE Trans. Image Process.*, vol. 29, pp. 4911–4926, 2020.

[8] Z. Lv, F. Wang, G. Cui, J. A. Benediktsson, T. Lei, and W. Sun, "Spatial–spectral attention network guided with change magnitude image for land cover change detection using remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–12, 2022, Art. no. 4412712.

[9] K. Xu, H. Huang, P. Deng, and Y. Li, "Deep feature aggregation framework driven by graph convolutional network for scene classification in remote sensing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 10, pp. 5751–5765, Oct. 2022.

[10] Z. Lv, H. Huang, L. Gao, J. A. Benediktsson, M. Zhao, and C. Shi, "Simple multiscale UNet for change detection with heterogeneous remote sensing images," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[11] J. M. Peña-Barragán, M. K. Ngugi, R. E. Plant, and J. Six, "Object-based crop identification using multiple vegetation indices, textural features and crop phenology," *Remote Sens. Environ.*, vol. 115, no. 6, pp. 1301–1316, Jun. 2011.

[12] M. Tan and Q. V. Le, "EfficientNetv2: Smaller models and faster training," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10096–10106.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[14] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.

[15] Y. Chen, J. Li, H. Xiao, X. Jin, S. Yan, and J. Feng, "Dual path networks," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 4470–4478.

[16] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 13733–13742.

[17] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 11976–11986.

[18] X. Li, C. Wen, Y. Hu, and N. Zhou, "RS-CLIP: Zero shot remote sensing scene classification via contrastive vision-language supervision," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 124, Nov. 2023, Art. no. 103497.

[19] C. Broni-Bediako, Y. Murata, L. H. B. Mormille, and M. Atsumi, "Searching for CNN architectures for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022, Art. no. 4701813.

[20] Y. Dai, F. Gieseke, S. Oehmcke, Y. Wu, and K. Barnard, "Attentional feature fusion," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3559–3568.

[21] X. Tang, Q. Ma, X. Zhang, F. Liu, J. Ma, and L. Jiao, "Attention consistent network for remote sensing scene classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 2030–2045, 2021.

[22] X. Huang, F. Liu, Y. Cui, P. Chen, L. Li, and P. Li, "Faster and better: A lightweight transformer network for remote sensing scene classification," *Remote Sens.*, vol. 15, no. 14, p. 3645, Jul. 2023.

[23] Z. Zheng, Y. Zhong, A. Ma, and L. Zhang, "FPGA: Fast patch-free global learning framework for fully end-to-end hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5612–5626, Aug. 2020.

[24] Q. Zhao, Y. Ma, S. Lyu, and L. Chen, "Embedded self-distillation in compact multibranch ensemble network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022, Art. no. 4506415.

[25] L. T. Yang, R. Zhao, D. Liu, W. Lu, and X. Deng, "Tensor-empowered federated learning for cyber-physical-social computing and communication systems," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 3, pp. 1909–1940, 2023.

[26] X. Zhang et al., "Remote sensing object detection meets deep learning: A metareview of challenges and advances," *IEEE Geosci. Remote Sens. Mag.*, vol. 11, no. 4, pp. 8–44, Dec. 2023.

[27] W. Zhang, P. Tang, and L. Zhao, "Remote sensing image scene classification using CNN-CapsNet," *Remote Sens.*, vol. 11, no. 5, p. 494, Feb. 2019.

[28] W. Wang, Y. Chen, and P. Ghamisi, "Transferring CNN with adaptive learning for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–18, 2022, Art. no. 5533918.

[29] F. Li, R. Feng, W. Han, and L. Wang, "High-resolution remote sensing image scene classification via key filter bank based on convolutional neural network," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 8077–8092, Nov. 2020.

[30] X. Wang, L. Duan, A. Shi, and H. Zhou, "Multilevel feature fusion networks with adaptive channel dimensionality reduction for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[31] S. Wang, Y. Guan, and L. Shao, "Multi-granularity canonical appearance pooling for remote sensing scene classification," *IEEE Trans. Image Process.*, vol. 29, pp. 5396–5407, 2020.

[32] Z. Zhao, J. Li, Z. Luo, J. Li, and C. Chen, "Remote sensing image scene classification based on an enhanced attention module," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 11, pp. 1926–1930, Nov. 2021.

[33] Q. Bi, H. Zhang, and K. Qin, "Multi-scale stacking attention pooling for remote sensing scene classification," *Neurocomputing*, vol. 436, pp. 147–161, May 2021.

[34] X. Zhang, S. Tian, G. Wang, X. Tang, J. Feng, and L. Jiao, "CAST: A cascade spectral-aware transformer for hyperspectral image change detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–14, 2023, Art. no. 5520714.

[35] R. Cao, L. Fang, T. Lu, and N. He, "Self-attention-based deep feature fusion for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 1, pp. 43–47, Jan. 2021.

[36] G. Zhang et al., "A multiscale attention network for remote sensing scene images classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 9530–9545, 2021.

[37] N. He, L. Fang, S. Li, J. Plaza, and A. Plaza, "Skip-connected covariance network for remote sensing scene classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 5, pp. 1461–1474, May 2020.

[38] N. He, L. Fang, S. Li, A. Plaza, and J. Plaza, "Remote sensing scene classification using multilayer stacked covariance pooling," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 12, pp. 6899–6910, Dec. 2018.

[39] J. Xie, N. He, L. Fang, and A. Plaza, "Scale-free convolutional neural network for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6916–6928, Sep. 2019.

[40] X. Zhang, X. Fan, G. Wang, P. Chen, X. Tang, and L. Jiao, "MFGNet: Multibranch feature generation networks for few-shot remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–13, 2023, Art. no. 5609613.

[41] Y. Bazi, M. M. Al Rahhal, H. Alhichri, and N. Alajlan, "Simple yet effective fine-tuning of deep CNNs using an auxiliary classification loss for remote sensing scene classification," *Remote Sens.*, vol. 11, no. 24, p. 2908, Dec. 2019.

[42] G. Cheng, C. Yang, X. Yao, L. Guo, and J. Han, "When deep learning meets metric learning: Remote sensing image scene classification via learning discriminative CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 5, pp. 2811–2821, May 2018.

[43] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11534–11542.

[44] L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "SimAM: A simple, parameter-free attention module for convolutional neural networks," in *Proc. ICML Deep Learn. (ICML)*, vol. 139, 2021, pp. 11863–11874.

[45] J. Ryu, E. Park, and H. Sung, "One-shot tuner for deep learning compilers," in *Proc. 31st ACM SIGPLAN Int. Conf. Compiler Construct.*, Mar. 2022, pp. 89–103.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.

[46] T. Chen et al., "Learning to optimize tensor programs," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2018, pp. 3393–3404.
[47] G.-S. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.
[48] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.
[49] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geograph. Inf. Syst.*, Nov. 2010, pp. 270–279.
[50] G. Sheng, W. Yang, T. Xu, and H. Sun, "High-resolution satellite scene classification using a sparse coding based multiple feature combination," *Int. J. Remote Sens.*, vol. 33, no. 8, pp. 2395–2412, Apr. 2012.
[51] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 197–209, Nov. 2018.
[52] Z. Huang et al., "CCNet: Criss-cross attention for semantic segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 6896–6908, Jun. 2023.
[53] Y.-E. Hou, K. Yang, L. Dang, and Y. Liu, "Contextual spatial-channel attention network for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.



Xiaohan Tu received the M.Sc. and Ph.D. degrees in computer science and technology from Hunan University, Changsha, China, in 2017 and 2021, respectively.

She is currently a Lecturer at Zhengzhou Police University, Zhengzhou, China. She has participated in the project of the National Natural Science Foundation of China: "CPS Instantiation-Research on the Smart Inspection Robot of Catenary." She has authored or coauthored more than ten research papers at IEEE international conferences or journals. Her research interests include cyber-physical systems, computer vision, and machine learning.

Dr. Tu is currently a reviewer for many journals.



Laurence Tianruo Yang (Fellow, IEEE) received the B.E. degree in computer science and technology from Tsinghua University, Beijing, China, in 1992, and the Ph.D. degree in computer science from the University of Victoria, Victoria, BC, Canada, in 2006.

He is currently a Professor with the Department of Computer Science, St. Francis Xavier University, Antigonish, NS, Canada. He has authored or coauthored more than 220 papers in various refereed journals (around 40% on top IEEE/ACM TRANSACTIONS AND JOURNALS, others mostly on Elsevier, Springer, and *Wiley Journals*). His research has been supported by the National Sciences and Engineering Research Council and the Canada Foundation for Innovation. His research interests include parallel and distributed computing, embedded and ubiquitous or pervasive computing, and big data.



Siping Liu received the M.Sc. and Ph.D. degrees in computer science and technology from Hunan University, Changsha, China, in 2017 and 2022, respectively.

He is currently an Assistant Researcher at Zhengzhou University, Zhengzhou, China. He has participated in the project of the National Natural Science Foundation of China: "Research on anomaly detection and adaptive network control in edge computing." His research interests include embedded and cyber-physical systems, cybersecurity, artificial intelligence, and machine learning.

Dr. Liu is currently a reviewer for many journals.



Renfa Li (Senior Member, IEEE) received the Ph.D. degree in electronic engineering from Huazhong University of Science and Technology, Wuhan, China, in 2002.

He is currently a Professor and the Chair of the embedded and cyber-physical systems with Hunan University, Changsha, China, where he is also the Chair of the Key Laboratory for Embedded and Cyber-Physical Systems. His major interests include computer architectures, embedded computing systems, cyber-physical systems, and Internet of things.

Prof. Li is a member of the council of CCF and a Senior Member of ACM.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:15:57 UTC from IEEE Xplore. Restrictions apply.